Advanced data management - assignment 1

Robin Staal & Ivar van der Spoel & Ivor Brouwer

September 2024

1 Introduction

This is the report for assignment 1 of advanced data management. In this assignment we benchmark multiple different database management systems (DMBSs): we will be benchmarking MonetDB [Tea24b], DuckDB [Tea24a] and PostgreSQL [Gro24]. To do this we use version 3 of the TPC-H [Cou24] benchmark which is a standard for commercial DBMS, specifically it is an OLAP benchmark. We include a section laying out the steps we took so it is possible to replicate our work. We show our results and discuss our findings.

2 Setting up the benchmarks

Below is an overview on how to setup the DMBSs with various dataset-sizes and how to use the queries provided for benchmarking. Most of the necessary files are included in the zip-file that can be downloaded from `https://homepages.cwi.nl/~manegold/tpc-h_v3.0.1.zip`.

2.1 Setting up a scale factor

The data where the queries are executed on can be found in the provided zip-file. There are however scale factors that determine the size of this data. For our benchmarks we will be using the SF-1, SF-3 and SF-10 scale factors, where the number implies how many GBs of data there is.

Getting the scale factors ready for a database is quite simple: in the supplied zip-file scale factor (SF) 1 and 3 are already present. These can be found under 'dbgen/SF-1' and 'dbgen/SF-3'. Within these folders a data-zip and the query results for that data is provided. This data-zip can be extracted to get the data that can later be loaded into the tables using sql.

The data from SF-10 that we used is not provided here but can be found on `https://homepages.cwi.nl/~manegold/SF-10.zip`, with the same directory structure as the other scale factors.

2.2 Setting up the databases

The below instructions are for setting up the databases on a windows 11 computer. All of these installations assume usage of Windows Powershell.

2.2.1 MonetDB

1. Install MonetDB for windows from the official website.
2. In Powershell, add the bin to the environment:

   ```
   $env:Path += ``;C:\<FILEPATH_TO_MONETDB>\MonetDB\MonetDB5\bin"
   ```

3. In Powershell, add the DotMonetDBFile to the environment:

   ```
   $env:DOTMONETDBFILE = "C:\<FILEPATH_TO_MONETDB>\MonetDB\MonetDB5\etc\
   .monetdb"
   ```

4. Start a MonetDB server by booting up the "MonetDB SQL Server" program.
5. By starting the server program, we automatically create the database 'demo'.
6. Create the tables for this database:

   ```
   mclient -U monetdb -d demo -i "C:\path\to\0-create_tables.sql"
   ```

7. Then, load the data in the tables. Make sure to adjust the file with the correct paths:

   ```
   mclient -U monetdb -d demo -i "C:\path\to\1-load_data.SF-1.sql"
   ```

8. Make sure to add the constraints:

   ```
   mclient -U monetdb -d demo -i "C:\path\to\2-add_constraints.sql"
   ```

2.2.2 DuckDB

1. Install DuckDB by opening powershell and downloading it through the Windows Packet Manager (winget):

   ```
   winget install DuckDB.cli
   ```

2. To create the database, we simply create the tables with constraints and this creates the necessary file:

   ```
   duckdb benchmark.duckdb
   ".read '.\<FILEPATH_TO_DUCKDB\duckdb\0-create_tables_with_constraints.sql'"
   ```

3. Then, load the data in the tables. Make sure to adjust the file with the correct paths:

```
duckdb benchmark.duckdb
".read '.\<FILEPATH_TO_DUCKDB\duckdb\1-load_data.sql'"
```

2.2.3 PostgreSQL

1. Install PostgreSQL for Windows from the official website.
2. In PowerShell, add the bin to the environment:

```
$env:Path += ``;C:\<FILEPATH_TO_POSTGRES>\PostgreSQL\17\bin"
```

3. The files supplied are not suited for PostgreSQL, as it does not accept an extra delimiter at the end. We fixed this by creating a Python script that removes all of these `|` delimiters from the end of the rows.
4. We open the PostgreSQL CLI by using:

```
psql -U postgres
```

5. Next, we create a database called `demo` by using the command:

```
CREATE DATABASE demo;
```

6. To create tables, we can use the SQL file supplied for DuckDB. We run the command:

```
psql -U postgres -d demo -f "C:\path\to\dbgen\DuckDB\
0-create_tables_with_constraints.sql
```

7. For loading the data into PostgreSQL, we had to make small changes to the `load_data.sql` file supplied. We had to change `COPY` to `\copy` and add the correct absolute paths to the SF-data files. To then load the data, we run:

```
psql -U postgres -d demo -f "C:\path\to\dbgen\scripts\
1-load_data-postgres.sql
```

2.3 Scripts to test and benchmark

Now that the DMBSs are set up and have the necessary data loaded, it is time to execute the queries to obtain our benchmark results.

To execute the benchmarks across different DBMSs, we developed several PowerShell test scripts designed to run the queries and produce results in a

specific format, allowing us to verify their accuracy. These scripts require modification to correctly reference the database input directory on each machine. Once the supplied TPC-H directory is unzipped, link the base input directory to the appropriate database.

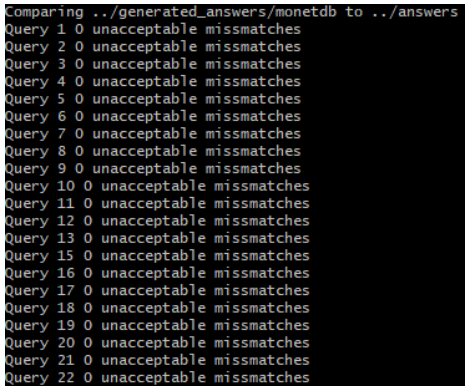
We also created powershell benchmark scripts for every DBMS that execute all queries ten times and output the execution time for the query to complete to a file. Similarly, the filepath must be set correctly for these scripts as well. For Postgres we used the built in timings functionality to test the runtime, for monetdb we use '**-t performance**' and for duckdb we use the built in '**.timer on**'.

It is also possible to run the queries individually by running the `q{x}.sql` files present in the monetDB and duckDB folders by using the CLI commands used in the earlier mentioned test scripts.

2.4 Verification of the results

In the provided zip-file, a `BASH` file is located in the 'check_answers' directory. We used this to verify whether our output coincided with the real answers provided in the 'answers' directory. Because of floating point rounding differences with the databases we use the provided `BASH` file to allow for small precision errors, to ensure these inevitable minuscule errors were not flagged as actual errors.

During the verification process we ran into some issues that are listed below. We were able to fix all these issues such that all outputs had 0 unacceptable mismatches. An example of the verification results of MonetDB can be seen in Figure 1.



```
Comparing ../generated_answers/monetdb to ../answers
Query 1 0 unacceptable missmatches
Query 2 0 unacceptable missmatches
Query 3 0 unacceptable missmatches
Query 4 0 unacceptable missmatches
Query 5 0 unacceptable missmatches
Query 6 0 unacceptable missmatches
Query 7 0 unacceptable missmatches
Query 8 0 unacceptable missmatches
Query 9 0 unacceptable missmatches
Query 10 0 unacceptable missmatches
Query 11 0 unacceptable missmatches
Query 12 0 unacceptable missmatches
Query 13 0 unacceptable missmatches
Query 15 0 unacceptable missmatches
Query 16 0 unacceptable missmatches
Query 17 0 unacceptable missmatches
Query 18 0 unacceptable missmatches
Query 19 0 unacceptable missmatches
Query 20 0 unacceptable missmatches
Query 21 0 unacceptable missmatches
Query 22 0 unacceptable missmatches
```

Figure 1: MonetDB output verification, query 14 is missing due to an overflow issue mentioned in Section 4.3.1

2.4.1 Sanitizing the outputs

For every DBMS there was a different way of sanitizing the output in order to make it verifiable. Below are the filters used. PostgreSQL needed the most filters just to implement the same behaviour as the other databases.

- **PostgreSQL:** `-q --pset=format=unaligned --pset=border=0 --pset=fieldsep="|" --pset="footer=off"`
- **MonetDB:** `-f csv+'|'`
- **DuckDB:** `-list`

2.4.2 UTF-16 output

One issue we ran into was that the standard output was in UTF-16, we did not know that this would affect the verification script and this caused it to flag multiple queries which output strings as unacceptable mismatches. We ended up fixing this by encoding all the output to UTF-8.

3 Own implementation of Q1 & Q6

We chose the C++ language for our own implementation of queries Q1 and Q6.

We start off by reading in all the rows from the given 'lineitem.tbl' file. We only store the data from the columns required by the selected query. We store all this required data in vectors, where every column has its own vector and the row id's are the indices of the vector.

We used the following compiler flags to optimize the program:

CXXFLAGS = -Wall -O3 -march=native -mtune=native -flto -funroll-loops -fomit-frame-pointer -ffast-math -funsafe-math-optimizations

3.1 Verification of the results

For verifying we utilize the same verification method used for the DBMS results in section 2.4. Below in figure 2 are the results of this verification.

```
$ ./pairs.sh ../generated_answers/cpp ../answers
Comparing ../generated_answers/cpp to ../answers
Query 1 0 unacceptable missmatches
Query 6 0 unacceptable missmatches
```

Figure 2: Both queries show 0 mismatches.

3.2 Q6

Running the query itself it was quite simple. The Q6 query only contained some regular filtering on the *shipdate*, *quantity* and *discount* columns. The query then only asks to calculate revenue on those filtered columns. We were able to simply implement this by iterating over all the vectors containing these columns and checking for those filters.

3.3 Q1

The Q1 query was a bit more complicated as it contained a 'group_by' and 'order_by' clause, on the *returnflag* and *lineflag* specifically. It also asked for more attributes to calculate for those filtered columns.

We implemented the 'group_by' using a map, with a pair containing the *returnflag* and *lineflag* as the key, and the required attributes in a C++ struct as the value. We chose to use a regular map here, since this would also automatically sort the pairs, thereby adhering to the 'order_by' clause immediately.

4 Benchmark results

The benchmarks were ran as explained in Section 2.3. We benchmarked three different DMBSs, namely: MonetDB, postgresql and DuckDB. Our benchmarks were run in 'hot' memory, which means the corresponding query had already been run before the benchmark so the data was already loaded in the memory. It is possible to also run benchmarks on 'cold' memory but this can cause the execution time to fluctuate more between runs, so it is often standard to run it in 'hot' memory. From the TPC-H benchmark (version 3.0.1) we used SF-1, SF-3 and SF-10 and executed these benchmarks on two different devices with the following hardware:

1. **Laptop**: Processor AMD Ryzen 5 5600H, Clock: 3.3GHz, L2 Cache: 3MB, L3 Cache: 16MB, 6 Core(s), 12 Logical Processor(s), 16GB RAM, SSD Storage 2200(MB/s) read.
2. **PC**: Processor Intel i7-10700, Clock: 2.90GHz, L2 Cache: 2MB, L3 Cache: 16MB, 8 Core(s), 16 Logical Processor(s), 16GB RAM, SSD Storage 2,611(MB/s) read.

For each benchmark (with unique (1) query, (2) hardware, (3) DBMS and (4) SF-size) we run 10 iterations to obtain a more accurate result. Our results are measured using **real time**, and shown in **milliseconds**.

4.1 Results from device 1

We show the results from our benchmarks in Tables 1, 2, 3 for SF-1, SF-3 and SF-10 respectively.

	Postgresql(ms)	MonetDB(ms)	DuckDB(ms)
Q1	1585.382	210.325	60.6
Q2	229.660	19.415	32.1
Q3	398.577	41.396	62.7
Q4	202.412	28.032	58.2
Q5	243.909	40.827	76.2
Q6	328.795	12.819	27.9
Q7	399.616	42.716	76.7
Q8	731.565	64.182	67.5
Q9	1056.086	65.865	168.8
Q10	719.673	37.076	124.8
Q11	201.245	12.556	14.9
Q12	485.339	26.027	45.9
Q13	1614.081	80.776	107.0
Q14	338.348	**x** (see Section 4.3.1)	49.9
Q15	740.085	19.554	52.0
Q16	337.285	49.707	43.1
Q17	**x** (see Section 4.3.3)	9.773	51.2
Q18	3758.341	72.662	116.7
Q19	461.607	23.329	73.7
Q20	**x** (see Section 4.3.3)	29.917	58.8
Q21	508.657	103.107	127.9
Q22	223.315	30.570	39.0

Table 1: Executing times with 22 unique queries. Three DBMSs are compared. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-1** directory

	Postgresql(ms)	MonetDB(ms)	DuckDB(ms)
Q1	4226.028	578.995	112.4
Q2	406.959	52.857	52.6
Q3	1069.783	103.145	115.8
Q4	2752.307	83.988	86.7
Q5	881.146	148.005	201.3
Q6	826.101	28.168	54.4
Q7	758.214	176.904	189.2
Q8	1513.417	233.462	157.4
Q9	2798.022	355.568	352.9
Q10	864.217	294.576	219.1
Q11	352.270	17.853	21.2
Q12	1269.527	53.638	89.6
Q13	4384.878	371.195	305.9
Q14	988.063	**x** (see Section 4.3.1)	95.7
Q15	592.034	12.861	42.3
Q16	752.055	90.641	76.2
Q17	**x** (see Section 4.3.3)	115.450	120.7
Q18	39390.761	139.352	303.9
Q19	1196.532	47.162	159.3
Q20	**x** (see Section 4.3.3)	50.533	128.3
Q21	1305.664	197.729	317.1
Q22	408.820	59.475	52.9

Table 2: Executing times with 22 unique queries. Three DBMSs are compared. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-3** directory

	Postgresql(ms)	MonetDB(ms)	DuckDB(ms)
Q1	20418.582	1762.943	**x**
Q2	1082.134	34.114	**x**
Q3	14828.066	248.498	**x**
Q4	67560.815	235.611	**x**
Q5	12707.830	484.438	**x**
Q6	11969.418	78.107	**x**
Q7	11518.854	592.994	**x**
Q8	12173.160	820.316	**x**
Q9	19346.016	2025.427	**x**
Q10	12848.394	714.063	**x**
Q11	1553.343	30.002	**x**
Q12	11645.930	127.931	**x**
Q13	15493.339	1616.799	**x**
Q14	12327.879	**x** (see Section 4.3.1)	**x**
Q15	24531.410	134.436	**x**
Q16	1850.980	278.355	**x**
Q17	**x** (see Section 4.3.3)	249.221	**x**
Q18	201188.153	392.781	**x**
Q19	13774.408	111.906	**x**
Q20	**x** (see Section 4.3.3)	141.259	**x**
Q21	16738.730	680.353	**x**
Q22	2114.341	155.202	**x**

Table 3: Executing times with 22 unique queries. Three DBMSs are compared. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-10** directory

	SF-1(ms)	SF-3(ms)
Q1	55.5121	174.641
Q6	17.9872	55.1166

Table 4: Executing times with Q1 and Q6. Tested on our own C++ implementation. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-1** directory

4.2 Results from device 2

We show the results from our benchmarks in Tables 5, 6, 7 for SF-1, SF-3 and SF-10 respectively.

	Postgresql(ms)	MonetDB(ms)	DuckDB(ms)
Q1	2061.332	143.717	40.400
Q2	249.088	11.330	24.600
Q3	686.845	31.054	44.200
Q4	237.270	24.958	35.700
Q5	263.378	33.973	44.600
Q6	431.109	7.534	17.400
Q7	562.306	37.339	47.500
Q8	841.236	44.125	48.000
Q9	1067.176	61.307	116.800
Q10	749.914	56.068	104.500
Q11	262.555	6.290	13.200
Q12	761.978	15.417	32.800
Q13	1252.644	75.643	88.600
Q14	463.292	**x** (see Section 4.3.1)	38.000
Q15	294.679	4.848	14.900
Q16	367.476	27.346	34.000
Q17	**x** (see Section 4.3.3)	28.009	43.200
Q18	2846.374	45.814	94.900
Q19	591.393	14.902	59.800
Q20	**x** (see Section 4.3.3)	16.558	47.500
Q21	641.353	57.362	105.200
Q22	248.873	17.008	33.400

Table 5: Executing times with 22 unique queries. Three DBMSs are compared. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-1** directory

	Postgresql(ms)	MonetDB(ms)	DuckDB(ms)
Q1	5355.862	478.797	86.800
Q2	571.556	21.273	45.900
Q3	1696.361	85.463	117.400
Q4	899.913	64.431	86.800
Q5	941.067	97.814	112.300
Q6	1116.653	21.265	54.100
Q7	1436.077	127.937	134.400
Q8	2100.534	142.387	116.900
Q9	2646.938	222.109	308.800
Q10	1787.680	170.182	200.300
Q11	416.624	9.630	21.800
Q12	1892.265	33.705	86.300
Q13	3341.655	250.678	357.600
Q14	1120.980	**x** (see Section 4.3.1)	111.100
Q15	758.071	9.806	43.433
Q16	774.932	68.038	79.700
Q17	**x** (see Section 4.3.3)	67.124	125.700
Q18	7056.074	110.064	320.200
Q19	1519.634	33.755	164.300
Q20	**x** (see Section 4.3.3)	39.051	103.600
Q21	1990.261	176.641	346.500
Q22	520.372	46.439	50.500

Table 6: Executing times with 22 unique queries. Three DBMSs are compared. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-3** directory

	Postgresql(ms)	MonetDB(ms)	DuckDB(ms)
Q1	17328.352	868.263	**x**
Q2	1601.778	56.559	**x**
Q3	10971.603	183.127	**x**
Q4	2659.522	281.959	**x**
Q5	5574.808	127.152	**x**
Q6	3995.510	22.966	**x**
Q7	11760.553	170.669	**x**
Q8	12536.235	185.772	**x**
Q9	16365.585	370.769	**x**
Q10	11800.880	259.077	**x**
Q11	1550.473	12.655	**x**
Q12	11972.757	48.454	**x**
Q13	13666.391	451.926	**x**
Q14	11940.183	**x** (see Section 4.3.1)	**x**
Q15	6733.488	14.690	**x**
Q16	2736.392	106.108	**x**
Q17	**x** (see Section 4.3.3)	111.206	**x**
Q18	72838.218	176.203	**x**
Q19	10565.508	45.716	**x**
Q20	**x** (see Section 4.3.3)	47.148	**x**
Q21	20361.525	220.202	**x**
Q22	4722.959	59.121	**x**

Table 7: Executing times with 22 unique queries. Three DBMSs are compared. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-10** directory

	SF-1(ms)	SF-3(ms)
Q1	521.702	1625.26
Q6	43.7148	131.117

Table 8: Execution times for Q1 and Q6. Tested on our own C++ implementation. Execution times are averaged over 10 iterations. The queries were executed on data from the **SF-1** directory.

4.3 Issues

While running these benchmarks we ran into a couple of issues, below is an elaboration on these issues.

4.3.1 MonetDB Q14 Overflow

For the 14th TPC-H query, MonetDB returns the following error instead of a query result:

```
#2024-09-25 16:36:49: DFLOWworker0003: convert_lng_lng:
ERROR: 22003!overflow in conversion to DECIMAL(18,10).
#2024-09-25 16:36:49: DFLOWworker0003: createExceptionInternal:
ERROR: SQLException:lng_2_lng:22003!overflow in conversion to DECIMAL(18,10).
```

This error occurs due to an overflow during conversion to the 'DECIMAL(18,10)' data type. If the value computed by the 'sum' in this query is too large, it can exceed 8 digits before the decimal point, and therefore the result cannot fit within the bounds of a 'DECIMAL(18,10)' type, causing an overflow.

We could fix this by rewriting the query to specify the type of this conversion, making the conversion explicit instead of implicit as it is now. This way we could specify the type to be of 'DECIMAL(18,10)' for example, leaving us with much more room before an overflow would occur.

4.3.2 Q15 Multiple outputs

Query 15 outputs 3 execution times per query, this happens because the query is made up of three parts: (1) creating a view, (2) running the query itself and (3) deleting the view. For our benchmark we chose to add these numbers up and then average them.

4.3.3 Q17 and Q20 Postgres

While running the benchmarks for Postgres we noticed that Q17 and Q20 would not finish on both machines, we did not have the option to keep them running for a long time and had to cut them short at around 45 minutes. We tried to test some optimizations listed below, but these did not improve execution time.

- shared_buffers = 4GB
- work_mem = 64MB
- maintenance_work_mem = 1GB
- effective_cache_size = 12GB
- max_parallel_workers_per_gather = 4
- max_parallel_workers = 8
- max_parallel_maintenance_workers = 4

4.3.4 SF-10 DuckDB

Unfortunately, we were not able to benchmark DuckDB on the SF-10 dataset. We ran into the following error on both devices:

```
TransactionContext Error: Failed to commit: failed to pin block
of size 256.0 KiB (12.5 GiB/12.5 GiB used)
```

We recognize that this problem can be fixed through the use of batching, but unfortunately we ran out of time take on this process further.

5 Discussion

In this section we analyze the performance of **PostresSQL**, **MonetDB** and **DuckDB** based on the result from Section 4.

5.1 DMBS comparisons

From Tables 1, 2, 5 and 6, it is abundantly clear that Postgres performs worst out of the three DMBSs. This lack in performance seems to be irregardless of what hardware it was run on or what query was executed, as in all measurements, Postgres had the slowest execution time. As we have addressed in Section 4.3.3, the sloth in execution time even caused us to not be able to run two of the queries.

We suspect this staggering difference is the cause of Postgres' lack of focus on benchmarks such as the TPC-H. As they mentioned on their Wikipedia [Posnd], as their aim is not on purposefully integrating code to optimize and account for specific commercial benchmarks.

Besides Postgres' performance we note that MonetDB and DuckDB fare quite a lot better against each other, yet MonetDB outperforms DuckDB in most configurations. We can see a few queries, namely Q1 & Q8, in which DuckDB manages a quicker execution time than MonetDB. For Q1, in which DuckDB outperformed MonetDB with a big margin, the main thing standing out is the abundance of aggregations in the query. We thus suspect this is what is making the difference in performance for this specific query.

5.2 Hardware based comparison

The two devices used for benchmarking the DBMSs differ significantly, most notably with one featuring an Intel CPU and the other an AMD Ryzen. To show how this affects the benchmarks, we create an easier visual in Table 3, where all results from the MonetDB benchmark are compared.

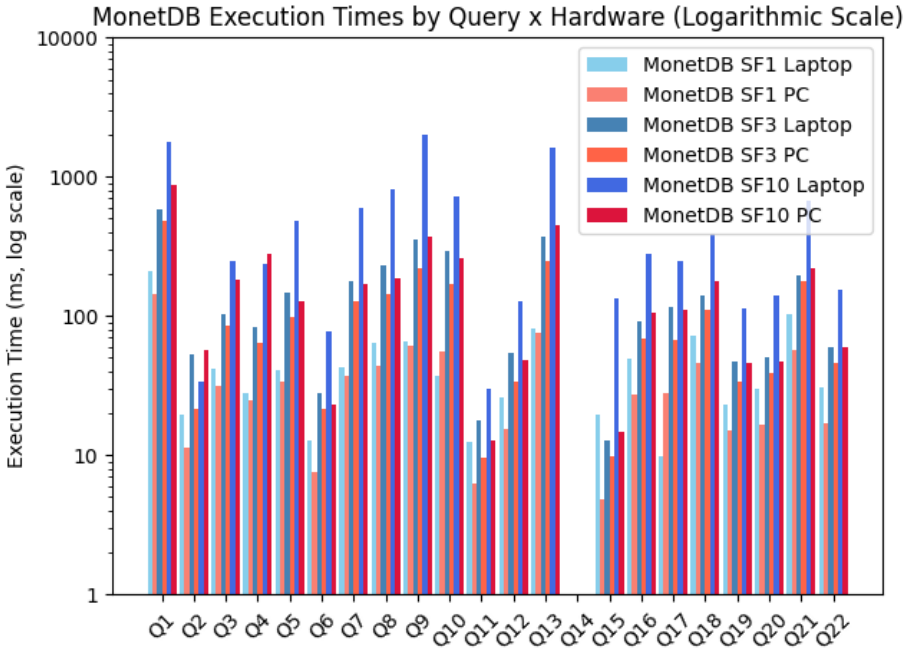


Figure 3: Executing times with 22 unique queries on MonetDB. Execution times are averaged over 10 iterations. Execution times of the PC vs Laptop hardware are compared on all scale factors.

In this visual it is clear that the PC hardware consistently scores a lower execution time, likely thanks to its higher SSD Storage.

5.3 C++ implementation vs Databases

In Tables 4 and 8 we find the execution times for our own C++ implementation of Q1 and Q6. When comparing these results to those of the DMBSs in Figure 4, we see that our implementation gets quite close to the other DMBSs, even consistently outperforming Postres. Specifically for the **Laptop** hardware our implementation is doing really well by managing to beat even MonetDB.

We suspect our implementation is able to do so well because it is not a perfect comparison: firstly, as we mentioned, we do not count the loading in off the data into our vectors. And secondly, because the query is already written into the code itself, it can be optimized by the compiler.

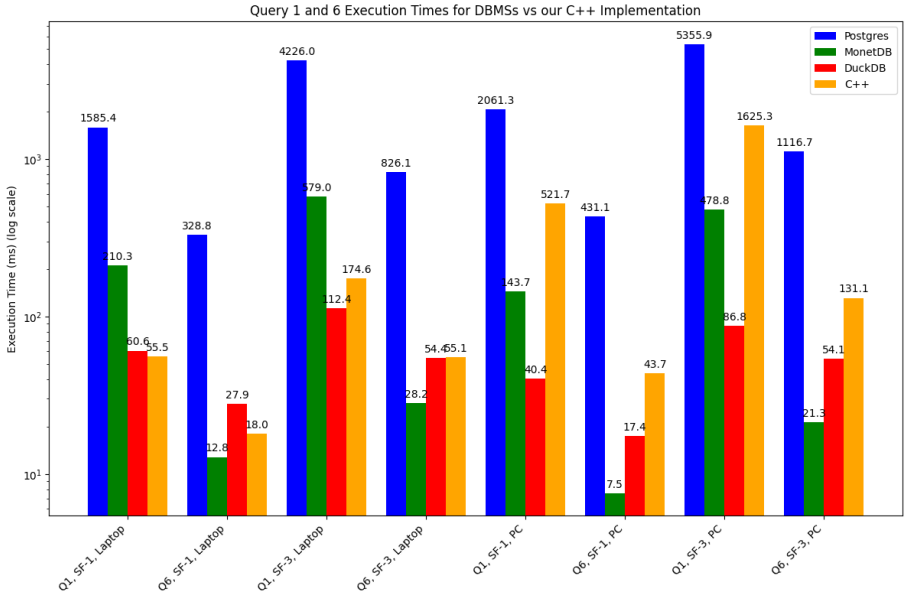


Figure 4: Executing times of Q1 and Q6, comparing DMBSs against custom C++ implemenation. Execution times are averaged over 10 iterations. Execution times of SF-1,3 are shown.

This result is not too surprising as our implementation was as simple as it gets without using special optimization for database querying.

6 Conclusions

Our results demonstrate significant performance differences between the systems, with MonetDB and DuckDB consistently outperforming PostgreSQL across most queries and configurations. PostgreSQL, while a versatile and widely-used DBMS, struggled with TPC-H queries, likely due to its lack of optimization for these specific OLAP workloads. MonetDB generally had the best performance, particularly on larger datasets, though DuckDB was competitive for certain queries.

Our hardware comparison showed that device specifications, especially SSD performance and CPU architecture, had a notable effect on execution times, highlighting the importance of hardware selection when evaluating database performance.

Additionally, the custom C++ implementation of queries 1 and 6 served as a useful comparison to the performance of DBMSs. Our simple implementations of these straightforward queries are able to compete in execution time against

MonetDB and DuckDB and outperform PostgreSQL by a significant amount .

Ultimately, our findings emphasize the need to choose the right database system depending on the specific workload, dataset size, and hardware environment. MonetDB and DuckDB are better suited for analytic workloads like TPC-H, whereas PostgreSQL may require additional tuning or may not be ideal for similar benchmarking without specific optimizations.

References

[Cou24] Transaction Processing Performance Council. Tpc-h benchmark. https://www.tpc.org/tpch/, 2024. Accessed: 2024-09-25.

[Gro24] PostgreSQL Global Development Group. Postgresql: The world's most advanced open source relational database. https://www.postgresql.org/, 2024. Accessed: 2024-09-25.

[Posnd] PostgreSQL Wiki Contributors. Tpc-h on postgresql, n.d. Accessed: 2024-09-29.

[Tea24a] DuckDB Development Team. Duckdb: An embeddable analytical database system. https://duckdb.org/, 2024. Accessed: 2024-09-25.

[Tea24b] MonetDB Team. Monetdb database management system. https://www.monetdb.org/, 2024. Accessed: 2024-09-25.